

CSAC PORTAL - EC2 DEPLOYMENT & GIT TROUBLESHOOT GUIDE

A quick-reference manual for handling Git authentication, database conflicts, and automated deployments

Issue 1: Git Pull Prompts for GitHub Credentials

Target Directory on EC2: `~/csac/csac_portal`

Description: When running `git pull origin main` on the EC2 server, Git prompts for your username and password, then aborts since GitHub has deprecated password authentication in favor of SSH Keys or Personal Access Tokens (PAT).

```
# Step 1: Check if you have an existing SSH key
cat ~/.ssh/id_rsa.pub

# Step 2: (If none exists) Generate a new SSH key
ssh-keygen -t rsa -b 4096 -C "your_email@example.com"
# (Press Enter to accept default path, leave passphrase blank)

# Step 3: Copy public key and add it to Repository -> Settings -> Deploy Keys
cat ~/.ssh/id_rsa.pub

# Step 4: Convert your repository remote URL from HTTPS to SSH
git remote set-url origin git@github.com:ghaneshsurya/csac.git

# Step 5: Test pull (will authenticate automatically without prompt)
git pull origin main
```

Explanation: This switches Git to use SSH Key authentication. Since your server's public key is registered as a 'Deploy Key' in your GitHub repository, GitHub securely authorizes pulls without prompting for any credentials.

Issue 2: Database File Conflict (Overwritten by Merge)

Target Directory on EC2: `~/csac/csac_portal`

Description: When running git pull or deploying, Git aborts with error: Your local changes to the following files would be overwritten by merge: csac_portal/db.sqlite3 (or db.sqlite3 if inside the directory).

```
# --- OPTION A: Overwrite Server DB with Git version (Recommended if no prod data yet) ---
# Step 1: Discard local changes to db.sqlite3
git checkout db.sqlite3

# Step 2: Re-run deploy script
./deploy.sh

# --- OPTION B: Keep Server DB Data ---
# Step 1: Backup production database
cp db.sqlite3 db.sqlite3.prod.bak

# Step 2: Revert git database tracking
git checkout db.sqlite3

# Step 3: Run deployment script
./deploy.sh

# Step 4: Restore production database
mv db.sqlite3.prod.bak db.sqlite3
```

Explanation: Git prevents pulls if a tracked file (like db.sqlite3) contains modifications on the server. Reverting the database (checkout) lets Git update it. If you have production data, backing it up first ensures your server database is preserved.

Issue 3: Divergent Branches Reconciliation Error

Target Directory on EC2: `~/csac/csac_portal`

Description: Git pull fails with the message fatal: Need to specify how to reconcile divergent branches. This occurs when server commits differ from or conflict with GitHub commits.

```
# Step 1: Fetch the latest references from remote GitHub repository
git fetch origin

# Step 2: Discard local server branch commits/divergences, resetting to remote
git reset --hard origin/main

# Step 3: Run the deploy script
./deploy.sh
```

Explanation: Using `git reset --hard origin/main` overrides the server's branch structure and matches the code on GitHub exactly. This is the recommended practice on production servers to prevent interactive merge conflicts.

Issue 4: Permission Denied when running deploy.sh

Target Directory on EC2: `~/csac/csac_portal`

Description: Running `./deploy.sh` fails with `-bash: ./deploy.sh: Permission denied` because the executable file permission flag was reset.

```
# Step 1: Add execution permissions (+x) to the deploy script
chmod +x deploy.sh

# Step 2: Execute the script
./deploy.sh
```

Explanation: Git resets or fails to track file executable permissions (+x) depending on the environment. Running `chmod +x` restores permissions so the system can run it as a command.
